RAG 技术详解与实战应用

第16讲：实践：打造具备宏观问答与图表生成功能的论文问答的RAG系统

目录

上节回顾

传统RAG的统计缺陷

- 无法直接处理结构化数据

它并不具备直接解析、理解和操作表格、数据库等结构化数据的能力。

- 缺乏动态计算能力：

许多结构化数据统计问题需要进行实时的计算和推理，传统的RAG模型只能基于已经检索到的信息生成答案，而无法在检索过程中执行这些复杂的计算任务。

依赖模块

- Text2SQL：

基于深度学习的 NLP 技术，能够将自然语言问题自动转换为 SQL 查询，无需用户掌握 SQL 语法。

- 意图识别模块

基于语言模型的意图识别器，用于根据用户提供的输入文本及对话上下文识别预定义的意图，并通过预处理和后处理步骤确保准确识别意图。

ChatBI

- ChatBI概念

聊天式商业智能，是一种结合了自然语言处理（NLP）、大语言模型（LLM）与数据分析能力的智能系统，允许用户 通过自然语言与数据对话，像和人聊天一样完成数据查询、报表生成、分析洞察等任务

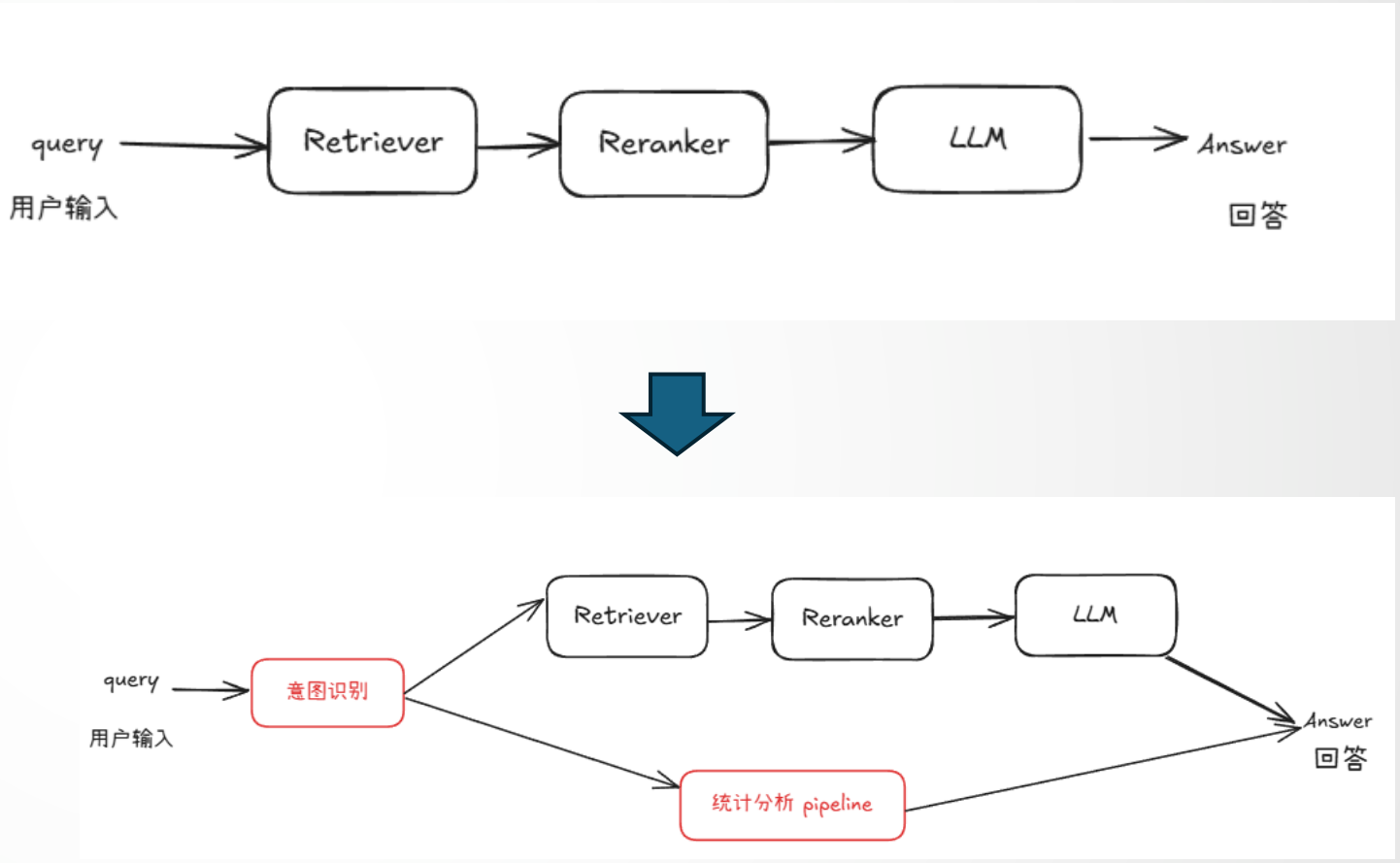
- Function Call

Function Call 是指在系统中调用预定义的函数，以实现特定功能

本节概览

本节目标：拓展论文问答系统，支持统计分析能力

- 在原有知识问答系统基础上，新增**统计分析 pipeline**，支持数据库查询、数据分析和图表生成
- 与原有 RAG pipeline 并行运行，通过**意图识别模块**动态调度，使系统兼具统计问答和知识问答的能力



本节内容结构：**数据准备 → 技术模块详解 → 系统搭建**

目录

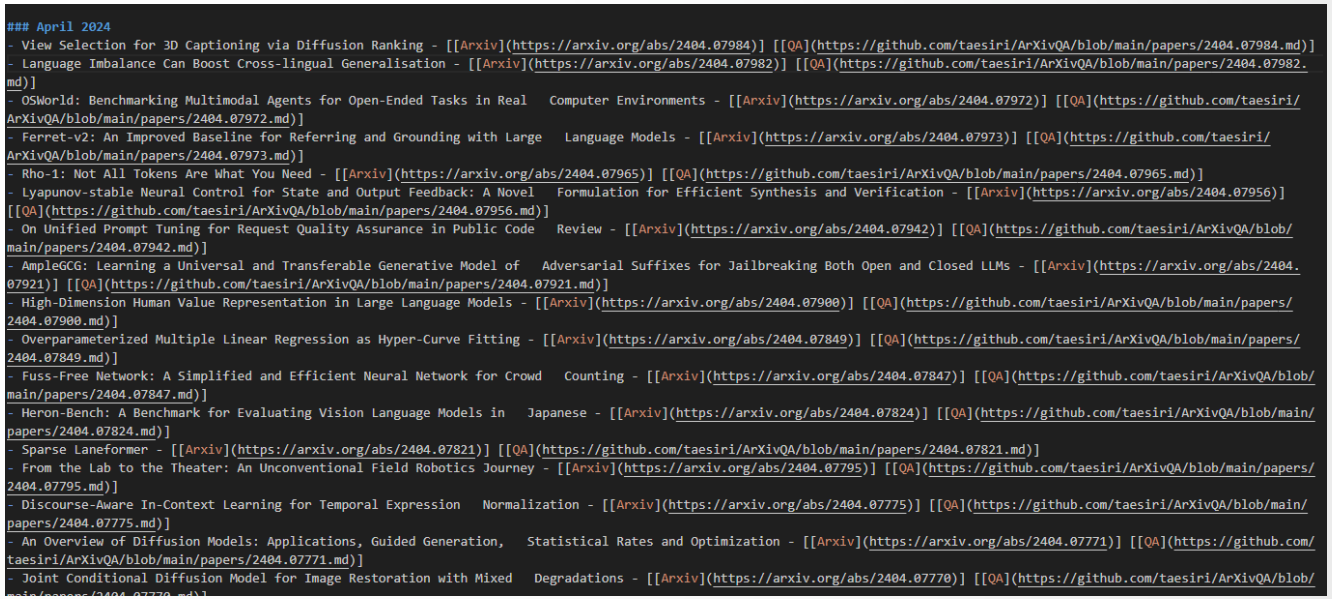
数据收集

以 ArXivQA 数据集为例，从 Papers-2024.md 中的100篇论文构建知识库和数据库，在准备数据之前需要先下载论文信息文件（内含论文链接），可以离线下载或者以下命令进行下载：

```
wget https://raw.githubusercontent.com/taesiri/ArXivQA/main/Papers-2024.md -O 2024.md
```

数据需求：

- **构建知识库：**下载论文原文用于知识问答
- **构建数据库：**抽取论文信息（title、author、subject…）构建数据库，用于统计问答



```
### April 2024
- View Selection for 3D Captioning via Diffusion Ranking - [[Arxiv](https://arxiv.org/abs/2404.07984)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07984.md)]
- Language Imbalance Can Boost Cross-lingual Generalisation - [[Arxiv](https://arxiv.org/abs/2404.07982)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07982.md)]
- OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real  Computer Environments - [[Arxiv](https://arxiv.org/abs/2404.07972)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07972.md)]
- Ferret-v2: An Improved Baseline for Referring and Grounding with Large  Language Models - [[Arxiv](https://arxiv.org/abs/2404.07973)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07973.md)]
- Rho-1: Not All Tokens Are What You Need - [[Arxiv](https://arxiv.org/abs/2404.07965)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07965.md)]
- Lyapunov-stable Neural Control for State and Output Feedback: A Novel  Formulation for Efficient Synthesis and Verification - [[Arxiv](https://arxiv.org/abs/2404.07956)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07956.md)]
- On Unified Prompt Tuning for Request Quality Assurance in Public Code  Review - [[Arxiv](https://arxiv.org/abs/2404.07942)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07942.md)]
- AmpleGCG: Learning a Universal and Transferable Generative Model of  Adversarial Suffixes for Jailbreaking Both Open and Closed LLMs - [[Arxiv](https://arxiv.org/abs/2404.07921)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07921.md)]
- High-Dimension Human Value Representation in Large Language Models - [[Arxiv](https://arxiv.org/abs/2404.07900)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07900.md)]
- Overparameterized Multiple Linear Regression as Hyper-Curve Fitting - [[Arxiv](https://arxiv.org/abs/2404.07849)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07849.md)]
- Fuss-Free Network: A Simplified and Efficient Neural Network for Crowd  Counting - [[Arxiv](https://arxiv.org/abs/2404.07847)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07847.md)]
- Heron-Bench: A Benchmark for Evaluating Vision Language Models in  Japanese - [[Arxiv](https://arxiv.org/abs/2404.07824)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07824.md)]
- Sparse Laneformer - [[Arxiv](https://arxiv.org/abs/2404.07821)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07821.md)]
- From the Lab to the Theater: An Unconventional Field Robotics Journey - [[Arxiv](https://arxiv.org/abs/2404.07795)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07795.md)]
- Discourse-Aware In-Context Learning for Temporal Expression  Normalization - [[Arxiv](https://arxiv.org/abs/2404.07775)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07775.md)]
- An Overview of Diffusion Models: Applications, Guided Generation,  Statistical Rates and Optimization - [[Arxiv](https://arxiv.org/abs/2404.07771)] [[QA](https://github.com/taesiri/ArXivQA/blob/main/papers/2404.07771.md)]
- Joint Conditional Diffusion Model for Image Restoration with Mixed  Degradations - [[Arxiv](https://arxiv.org/abs/2404.07770)] [[QA](https://github.com/taesiri/ArXivQA/blob/
```

数据收集

数据处理：

- **构建知识库：**下载论文到指定目录
- **构建数据库：**抽取论文信息，创建数据库文件(**.db)

可实现爬虫脚本同时执行以上两项任务

```
1.  def spider(url, save_dir='./rag_data/papers' ):
2.      ''' 下载论文并且抽取论文信息 '''
3.      response = requests.get(url)

4.      # 检查请求是否成功
5.      res = {}
6.      if response.status_code == 200:
7.          # 解析网页内容
8.          soup = BeautifulSoup(response.text,
'html.parser' )
9.          res  = …
10.         # 保存论文
11.         file_path = os.path.join(save_dir, res['title']
+ '.pdf' )
12.         with open(file_path, 'wb') as f:
13.             f.write(response.content)
14.         print(f"已保存 PDF 到 {file_path}")
15.         return res

16.     else:
17.         print(f"网页加载失败，状态码: {response.status_code}")
```

论文文件路径

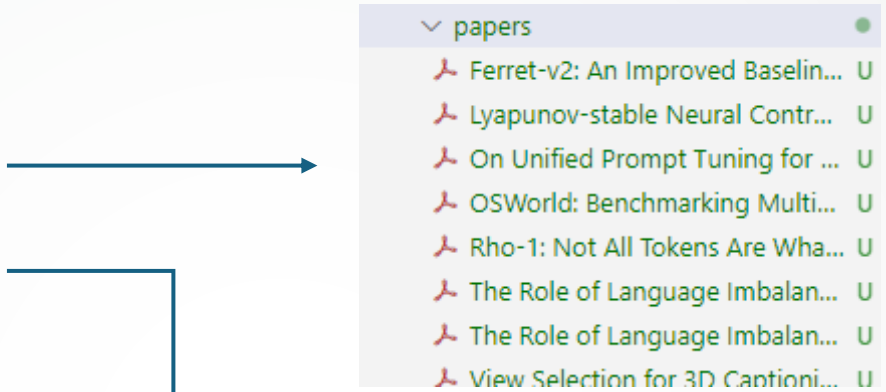
```
1.  def write_into_table():
2.      ''' 创建数据库、定义数据表、写入数据 '''
3.      paper = …
4.      # 连接到 SQLite 数据库（如果数据库不存在，它会被创建）
5.      conn = sqlite3.connect('papers.db' )
6.      cursor = conn.cursor()

7.      # 创建表格
8.      cursor.execute( ‘ “
9.      CREATE TABLE IF NOT EXISTS papers (
10.         id INTEGER PRIMARY KEY AUTOINCREMENT,
11.         title TEXT,
12.         author TEXT,
13.         subject TEXT
14.     )
15.     ' “ )

16.     # 插入数据
17.     for paper in data:
18.         cursor.execute(' “
19.         INSERT INTO papers (title, author, subject)
20.         VALUES (?, ?, ?)
21.         ''', (paper['title'], paper['author'],
paper['subject']))

22.     # 提交并关闭连接
23.     conn.commit()
24.     conn.close()
```

数据库存储路径

数据收集

数据准备完成

- 下载论文至./rag/papers 文件夹下
- 数据库文件 papers.db 构建完成



	title	author	subject
1	View Selection for 3D Captioning via Diffusion Ranking	Tiange Luo;Justin Johnson;Honglak Lee	Computer Vision and Pattern Recognition
2	The Role of Language Imbalance in Cross-lingual Gener...	Anton Schäfer;Shauli Ravfogel;Thomas Hofmann;Tiago P...	Computation and Language (cs.CL); Machi
3	OSWorld: Benchmarking Multimodal Agents for Open-E...	Tianbao Xie;Danyang Zhang;Jixuan Chen;Xiaochuan Li;Si...	Artificial Intelligence (cs.AI); Computation
4	Ferret-v2: An Improved Baseline for Referring and Groun...	Haotian Zhang;Haoxuan You;Philipp Dufter;Bowen Zhan...	Computer Vision and Pattern Recognition
5	Rho-1: Not All Tokens Are What You Need	Zhenghao Lin;Zhibin Gou;Yeyun Gong;Xiao Liu;Yelong S...	Computation and Language (cs.CL); Artific
6	Lyapunov-stable Neural Control for State and Output Fe...	Lujie Yang;Hongkai Dai;Zhouxing Shi;Cho-Jui Hsieh;Russ ...	Machine Learning (cs.LG); Artificial Intellig
7	On Unified Prompt Tuning for Request Quality Assuranc...	Xinyu Chen;Lin Li;Rui Zhang;Peng Liang	Software Engineering (cs.SE); Artificial Inte
8	AmpleGCG: Learning a Universal and Transferable Gener...	Zeyi Liao;Huan Sun	Computation and Language (cs.CL)
9	High-Dimension Human Value Representation in Large L...	Samuel Cahyawijaya;Delong Chen;Yejin Bang;Leila Khalat...	Computation and Language (cs.CL); Artific
10	Overparameterized Multiple Linear Regression as Hyper...	E. Atza;N. Budko	Machine Learning (stat.ML); Machine Lear
11	The Effectiveness of a Simplified Model Structure for Cro...	Lei Chen;Xinghang Gao;Fei Chao;Xiang Chang;Chih Min ...	Computer Vision and Pattern Recognition
12	Heron-Bench: A Benchmark for Evaluating Vision Langu...	Yuichi Inoue;Kento Sasaki;Yuma Ochi;Kazuki Fujii;Kotaro ...	Computer Vision and Pattern Recognition
13	Sparse Laneformer	Ji Liu;Zifeng Zhang;Mingjie Lu;Hongyang Wei;Dong Li;Yil...	Computer Vision and Pattern Recognition
14	From the Lab to the Theater: An Unconventional Field R...	Ali Imran;Vivek Shankar Varadharajan;Rafael Gomes Bra...	Robotics (cs.RO)
15	Discourse-Aware In-Context Learning for Temporal Expr...	Akash Kumar Gautam;Lukas Lange;Jannik Strötgen	Computation and Language (cs.CL); Artific
16	An Overview of Diffusion Models: Applications, Guided ...	Minshuo Chen;Song Mei;Jianqing Fan;Mengdi Wang	Machine Learning (cs.LG); Statistics Theory
17	Joint Conditional Diffusion Model for Image Restoration...	Yufeng Yue;Meng Yu;Luojie Yang;Yi Yang	Computer Vision and Pattern Recognition
18	RMAFF-PSN: A Residual Multi-Scale Attention Feature F...	Kai Luo;Yakun Ju;Lin Qi;Kaixuan Wang;Junyu Dong	Computer Vision and Pattern Recognition
19	Generating Synthetic Satellite Imagery With Deep-Learni...	Tuong Vy Nguyen;Alexander Glaser;Felix Biessmann	Computer Vision and Pattern Recognition
20	Mitigating Vulnerable Road Users Occlusion Risk Via Col...	Vincent Albert Wolff;Edmir Xhoxhi	Networking and Internet Architecture (cs.N
21	Reframing the Mind-Body Picture: Applying Formal Syst...	Ryan Williams	Artificial Intelligence (cs.AI); Neurons and

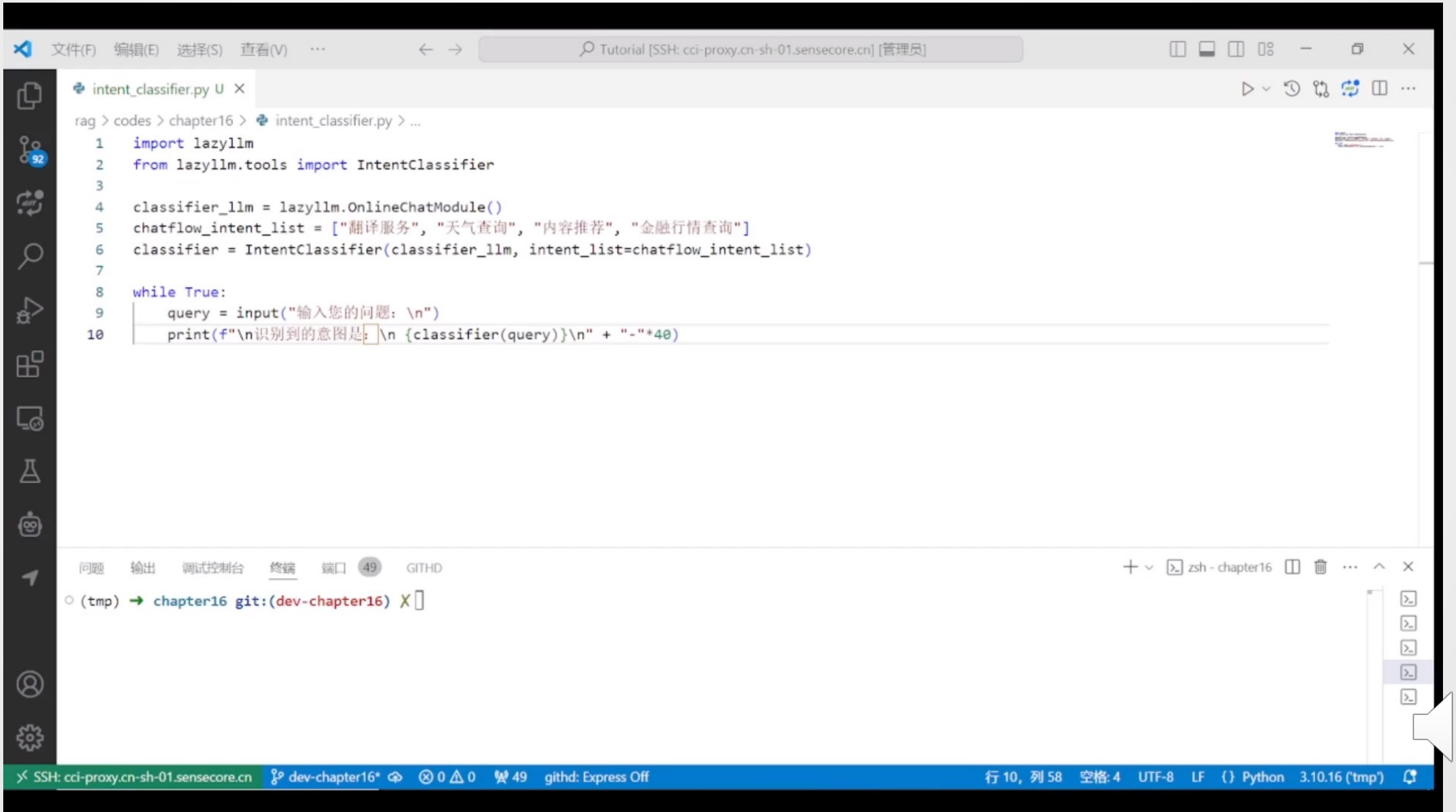
目录

意图识别

- 该模块的主要职责是分析用户的输入内容，判断其意图属于传统的知识问答，还是涉及到结构化数据处理的统计问答。
- LazyLLM内置基于语言模型的意图分类工具 IntentClassifier ，用于根据用户提供的输入文本及对话上下文识别预定义的意图。

```
1.  import lazyllm
2.  from lazyllm.tools import IntentClassifier

3.  classifier_llm = lazyllm.OnlineChatModule(source="qwen")
4.  chatflow_intent_list = ["翻译服务", "天气查询", "内容推荐", "金融行情查询”]
5.  classifier = IntentClassifier(classifier_llm, intent_list=chatflow_intent_list)
6.  classifier.start()

7.  print(classifier( ‘今天天气怎么样'))
# 输出 >>> 天气查询
```

意图识别


intent_classifier.py U
rag > codes > chapter16 > intent_classifier.py > ...
1 import lazyllm
2 from lazyllm.tools import IntentClassifier
3
4 classifier_llm = lazyllm.OnlineChatModule()
5 chatflow_intent_list = ["翻译服务", "天气查询", "内容推荐", "金融行情查询"]
6 classifier = IntentClassifier(classifier_llm, intent_list=chatflow_intent_list)
7
8 while True:
9 query = input("输入您的问题: \n")
10 print(f"\n识别到的意图是: \n {classifier(query)}\n" + "-"*40)
问题 输出 调试控制台 终端 端口 49 GITHD
zsh - chapter16
(tmp) → chapter16 git:(dev-chapter16) X
SSH: cci-proxy.cn-sh-01.sensecore.cn
dev-chapter16*
0 0
49
githd: Express Off
行 10，列 58
空格: 4
UTF-8
LF
Python
3.10.16 ('tmp')

意图识别 – 高级用法

- 通过with语法，使得意图识别模块能够直接跟随具体的指令

```
base = TrainableModule('internlm2-chat-7b’ )
with IntentClassifier(base) as ic:
    ic.case['聊天', base]
    ic.case['语音识别', TrainableModule('SenseVoiceSmall’ )]
    ic.case['图片问答', TrainableModule('Mini-InternVL-Chat-2B-V1-5').deploy_method(deploy.LMDeploy)]
    ic.case['画图', pipeline(base.share().prompt(painter_prompt),
            TrainableModule('stable-diffusion-3-medium’ ))]
    ic.case['生成音乐', pipeline(base.share().prompt(musician_prompt), TrainableModule('musicgen-small’ ))]
    ic.case['文字转语音', TrainableModule('ChatTTS’ )]
WebModule(ic, history=[base], audio=True, port=8847).start().wait()
```

- 使用方式有case[a, b] case[a: b] 和 case[a::b]三种，可以根据个人习惯选择使用，三者无区别：

```
with IntentClassifier(base) as ic:
    ic.case['聊天', base]
    ic.case['语音识别': TrainableModule('SenseVoiceSmall’ )]
    ic.case['图片问答'::TrainableModule('Mini-InternVL-Chat-2B-V1-5').deploy_method(deploy.LMDeploy)]
```

SQL Manager

数据库管理模块 —— SQL Manager

- SQL Manager 基于Python 中主流的 ORM 框架——SQLAlchemy。其提供了统一的数据库操作接口，不仅支持本地数据库，也支持各种主流云数据库，极大地提升了数据库交互的灵活性和可扩展性。
- 通过SQLManager，用户可以非常简单地连接本地或云上的数据库，只需在配置中指定数据库地址、账号密码等信息，就可以一键启动数据库交互能力，统一调用 SQL 查询、执行 SQL-call 相关操作。

```
1.  from lazyllm.tools import SqlManager
2.  # 连接本地数据库
3.  sql_manager = SqlManager("sqlite", None, None, None, None,
4.    db_name="papers.db",   tables_info_dict=table_info)
5.  # 连接远程数据库
6.  # sql_manager = SqlManager(type="PostgreSQL", user="",
7.        password="", host="",port="", name="",)

8.  # 查询所有论文数量
9.  res = sql_manager.execute_query("select count(*) as total_papser from papers;")
>>> [{"total_papser": 100}]

# 查询带有 LLM 的论文标题
10.  res = sql_manager.execute_query("select title from papers where title like '%LLM%'")
>>> [{"title": "AmpleGCG: Learning a Universal and Transferable Generative Model of Adversarial Suffixes
for Jailbreaking Both Open and Closed LLMs"}, {"title": "Learning to Localize Objects Improves Spatial
Reasoning in Visual-LLMs"}, {"title": "LLMs in Biomedicine: A study on clinical Named Entity Recognition"}
{"title": "VLLMs Provide Better Context for Emotion Understanding Through Common Sense Reasoning"}]
```

```
table_info = {
    "tables": [{
        "name": "papers",
        "comment": "论文数据",
        "columns": [
            {
                "name": "id",
                "data_type": "Integer",
                "comment": "序号",
                "is_primary_key": True,
            },
            {"name": "title", "data_type": "String", "comment": "标题"},
            {"name": "author", "data_type": "String", "comment": "作者"},
            {"name": "subject", "data_type": "String", "comment": "领域"},
        ],
    }]
}
```

传入数据库基本信息字典（表名、列名等）

SQL Call

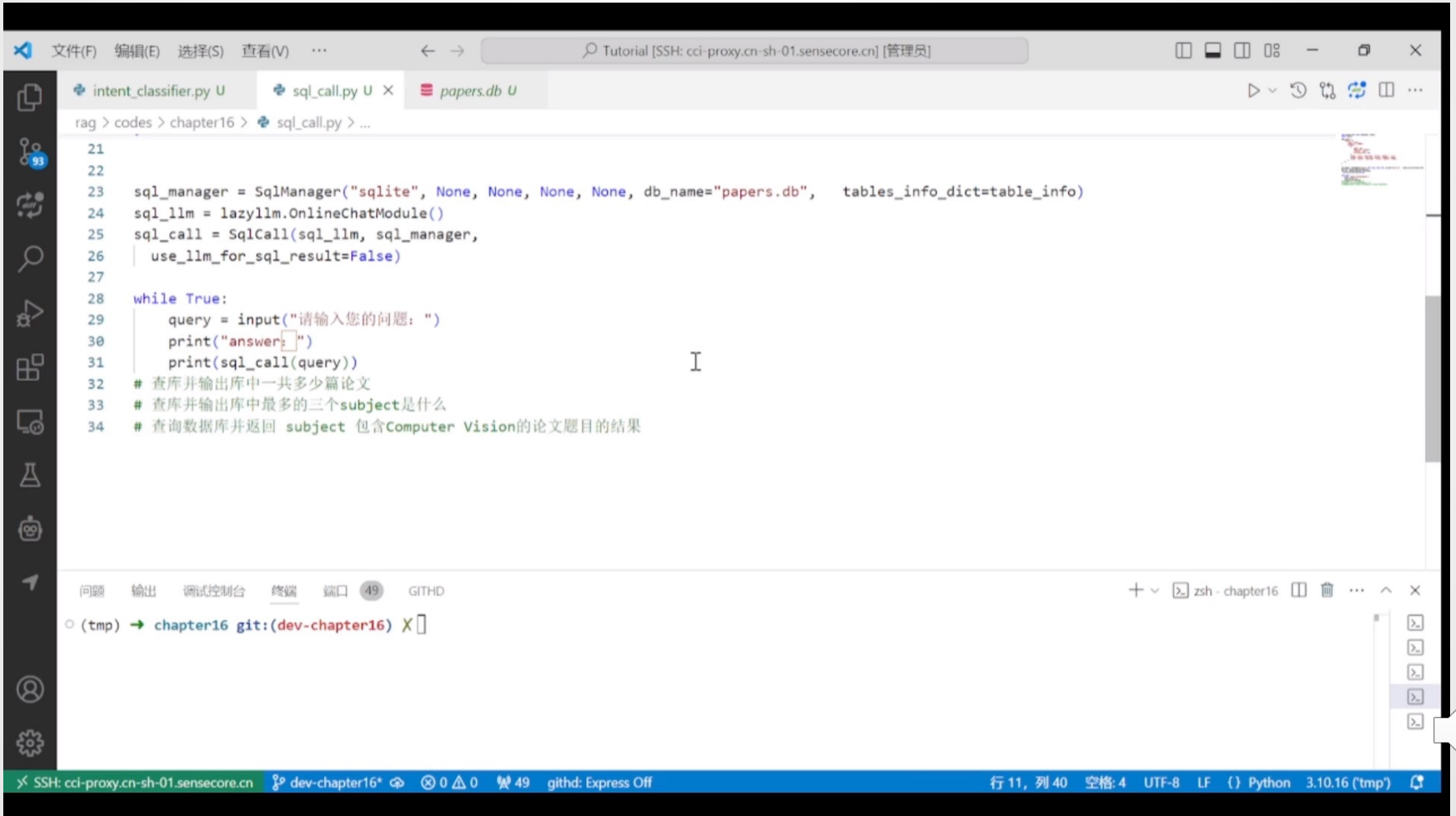
统计问答的第一环：数据库查询&数据获取 —— SQL Call （基于SQL Manager）

- 通过指定数据库名称和数据表信息创建 sql_manager，同时实例化一个 LLM。
- 将二者传入以构建一个 SQL Call 工作流，该工作流支持将自然语言查询转化为 SQL 语句并执行查询，最终返回结果。

```
1.  from lazyllm.tools import SqlManger, SqlCall

2.  sql_manager = SqlManager("sqlite", None, None, None, None, db_name="papers.db",
3.       tables_info_dict=table_info)
4.  sql_llm = lazyllm.OnlineChatModule()
5.  sql_call = SqlCall(sql_llm, sql_manager,
6.       use_llm_for_sql_result=False)

7.  query = "库中一共多少篇文章"
8.  print(sql_call(query))
>>> [{"total_papers": 100}]
```

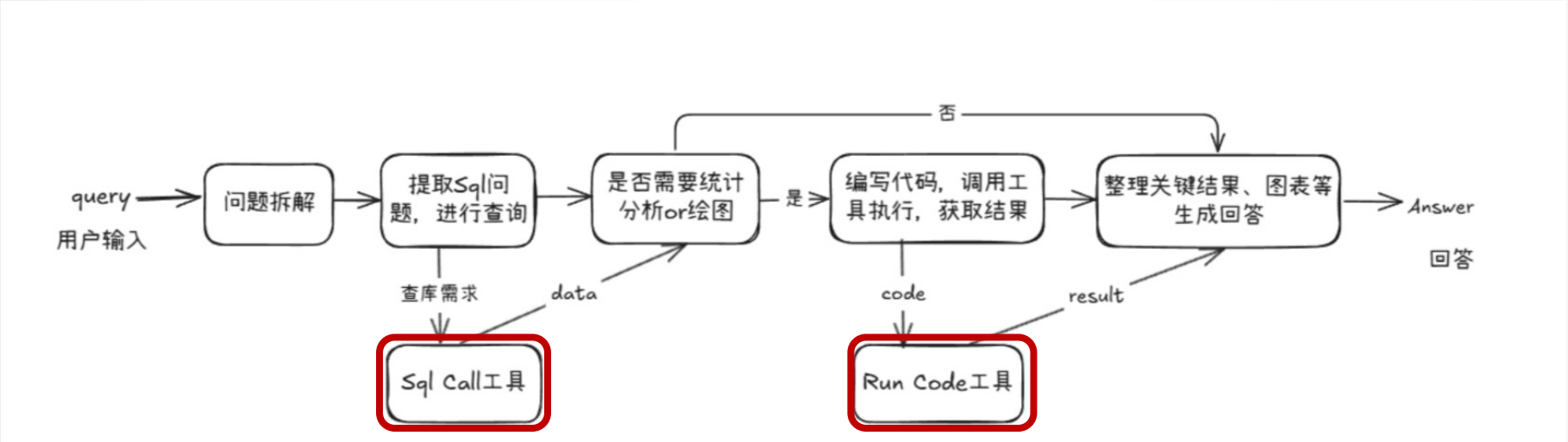
传入上一小节构建的
数据库路径

SQL Call


文件(F) 编辑(E) 选择(S) 查看(V) ···
Tutorial [SSH: cci-proxy.cn-sh-01.sensecore.cn] [管理员]
intent_classifier.py U
sql_call.py U
papers.db U
rag > codes > chapter16 > sql_call.py > ...
21
22
23 sql_manager = SqlManager("sqlite", None, None, None, None, db_name="papers.db", tables_info_dict=table_info)
24 sql_llm = lazyllm.OnlineChatModule()
25 sql_call = SqlCall(sql_llm, sql_manager,
26 use_llm_for_sql_result=False)
27
28 while True:
29 query = input("请输入您的问题：")
30 print("answer:")
31 print(sql_call(query))
32 # 查库并输出库中一共多少篇论文
33 # 查库并输出库中最多的三个subject是什么
34 # 查询数据库并返回 subject 包含Computer Vision的论文题目的结果
问题 输出 调试控制台 终端 端口 49 GITHD
zsh - chapter16
(tmp) → chapter16 git:(dev-chapter16) X
SSH: cci-proxy.cn-sh-01.sensecore.cn
dev-chapter16*
0 0
49
githd: Express Off
行 11，列 40
空格: 4
UTF-8
LF
{} Python
3.10.16 ('tmp')

统计分析Agent

- 基于sql call构建一个具备 **数据查询、统计分析与图表绘制能力** 的 Agent
- 根据用户问题自动完成 **数据库查询 → 数据分析 → 图表生成 → 结果汇总** 的完整闭环流程



- 首先定义两个工具（Tool），分别用于 **SQL 查询** 和 **代码执行**，并将它们作为可选工具提供给 Agent，支持其在执行任务过程中完成数据库查询和统计分析。
- 这两个工具配合使用，使 Agent 能够动态完成从数据获取到分析执行的全流程任务。

统计分析Agent

- Tool1：run_sql_query

  方法基于上一节构建的 SQL Call 工作流封装而成，Agent 可在从用户查询中解析出 SQL 相关需求后调用该方法，获取结构化查询结果。

```
@fc_register("tool")
def run_sql_query(query: str):
    """
    Translates a natural language data request into SQL, executes it, and returns
structured results.

    Args:
        query (str): A natural language description of the desired database query.

    Returns:
        list[dict]: A list of records returned from the SQL query, where each record is
represented as a dictionary.
    """
    sql_manager = SqlManager("sqlite", None, None, None, None, db_name="papers.db",
tables_info_dict=table_info)
    sql_llm = lazyllm.OnlineChatModule(stream=False)
    sql_call = SqlCall(sql_llm, sql_manager, use_llm_for_sql_result=False)
    return sql_call(query)
```

统计分析Agent

- Tool2：run_code

用于接收并执行大模型生成的统计分析代码。该方法会返回执行结果为大模型下一步的决策和行动提供反馈依据

```
@fc_register("tool")
def run_code(code: str):
    """
    Run the given code in a separate thread and return the result.

    Args:
        code (str): code to run.

    Returns:
        dict: {
            "text": str,
            "error": str or None,
        }
    """
    result = {
        "text": "",
        "error": None,
    }

    def code_thread():
        …
    thread = threading.Thread(target=code_thread)
    thread.start()
    thread.join(timeout=10)

    …
    return result
```

统计分析Agent

接下来实现agent定义，使用lazyllm内置的ReactAgent工具，将前文中定义的两个工具传入tools参数，即可实现模型自动调用。

ReactAgent是按照 Thought->Action->Observation->Thought...->Finish 的流程一步一步的通过LLM和工具调用来显示解决用户问题的步骤，最后输出最终答案。

```
from lazyllm.tools.agent import ReactAgent

agent = ReactAgent(
        llm=lazyllm.OnlineChatModule(stream=False),
        tools=['run_code', 'run_sql_query'],
        return_trace=True,
        max_retries=3,
    )
```

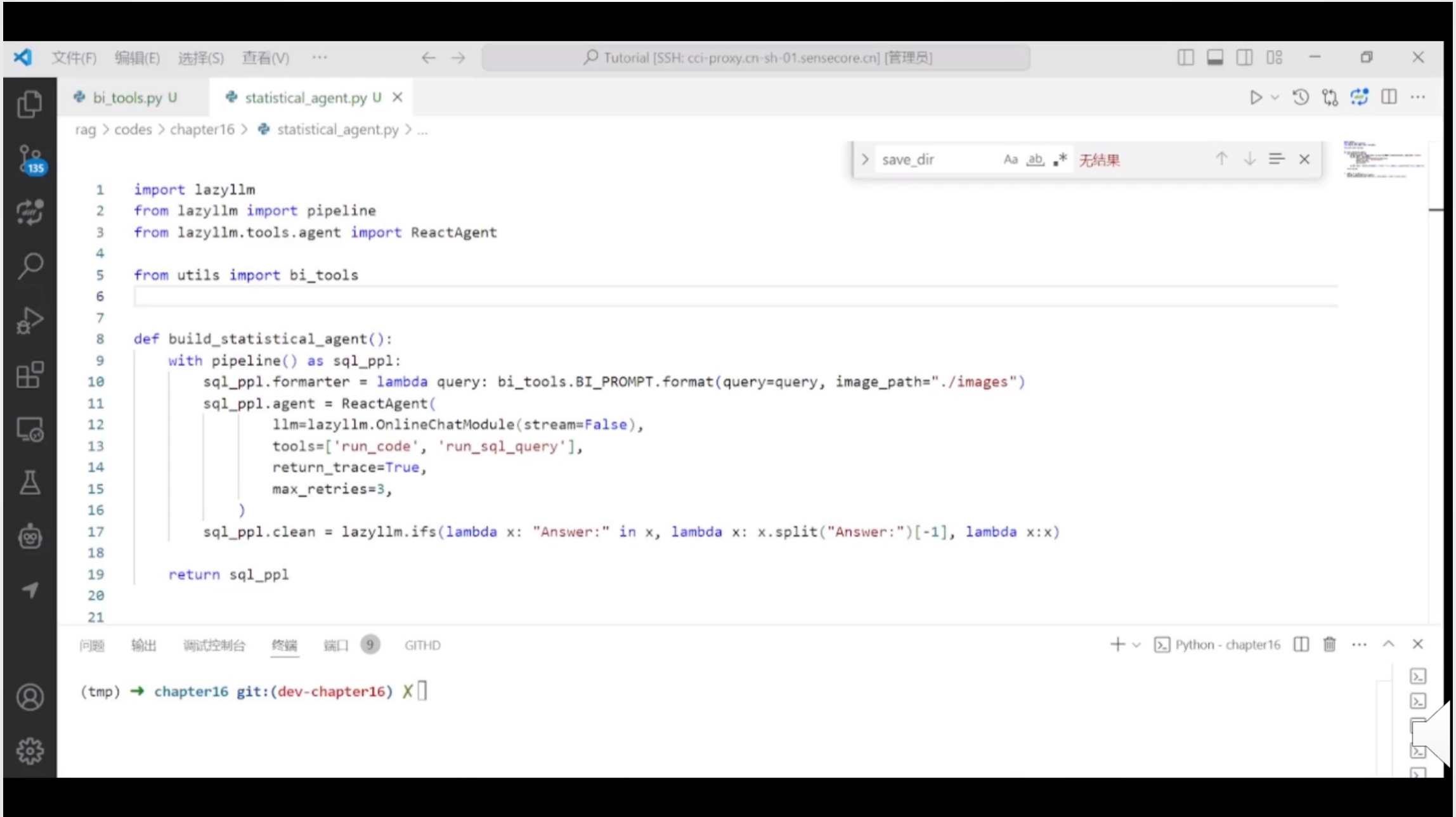
传入已通过@fc_register("tool")注册的工具的函数名称

统计分析Agent

完整代码如下

```
1.  with pipeline() as sql_ppl:
2.      sql_ppl.formarter = lambda query: sql_prompt.format(query=query,
image_path=IMAGE_PATH)
3.      sql_ppl.agent = ReactAgent(
4.              llm=lazyllm.OnlineChatModule(stream=False),
5.              tools=['run_code', 'run_sql_query' ],
6.              return_trace=True,
7.              max_retries=3,
8.          )
9.     sql_ppl.clean_output = lazyllm.ifs(lambda x: "Answer:" in x, lambda x:
10.         x.split("Answer:")[-1], lambda x:x)

11. if __name__ == '__main__' :
12.     lazyllm.WebModule(sql_ppl, port=23456,
static_paths=image_save_path).start().wait()
```

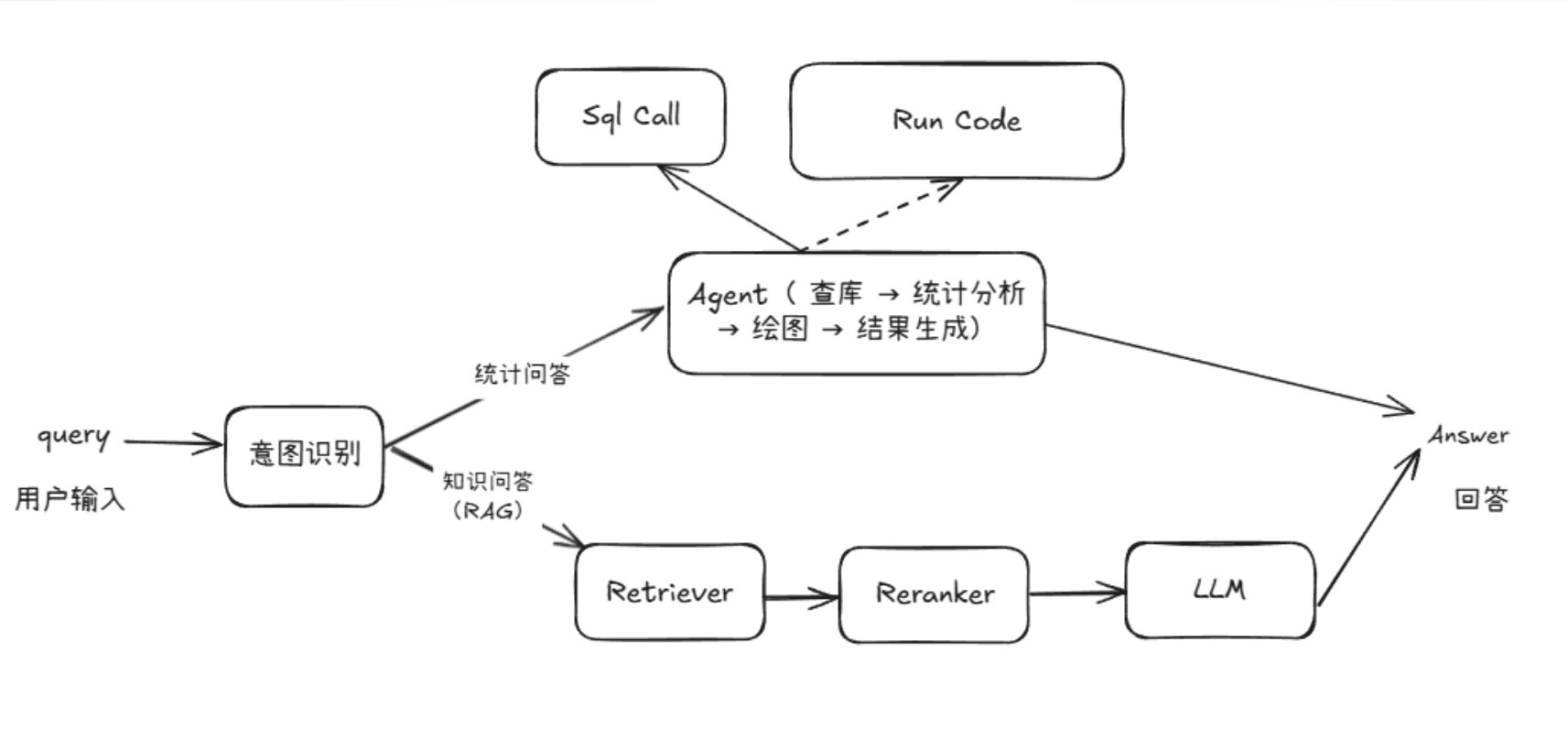
运行效果


文件(F) 编辑(E) 选择(S) 查看(V) ···
Tutorial [SSH: cci-proxy.cn-sh-01.sensecore.cn] [管理员]
bi_tools.py U
statistical_agent.py U
rag > codes > chapter16 > statistical_agent.py > ...
save_dir Aa ab .* 无结果
import lazyllm
from lazyllm import pipeline
from lazyllm.tools.agent import ReactAgent
from utils import bi_tools
def build_statistical_agent():
with pipeline() as sql_ppl:
sql_ppl.formarter = lambda query: bi_tools.BI_PROMPT.format(query=query, image_path="./images")
sql_ppl.agent = ReactAgent(
llm=lazyllm.OnlineChatModule(stream=False),
tools=['run_code', 'run_sql_query'],
return_trace=True,
max_retries=3,
)
sql_ppl.clean = lazyllm.ifs(lambda x: "Answer:" in x, lambda x: x.split("Answer:")[-1], lambda x:x)
return sql_ppl
问题 输出 调试控制台 终端 端口 9 GITHD
Python - chapter16
(tmp) → chapter16 git:(dev-chapter16) X

目录

系统架构与流程

上一节已经定义好了支持统计分析能力的pipeline，我们在这里结合论文问答RAG实现一个完整的问答系统，使其同时具备知识问答和统计问答的能力，并且还可以绘制相应的图表。



代码实现

完整代码如下

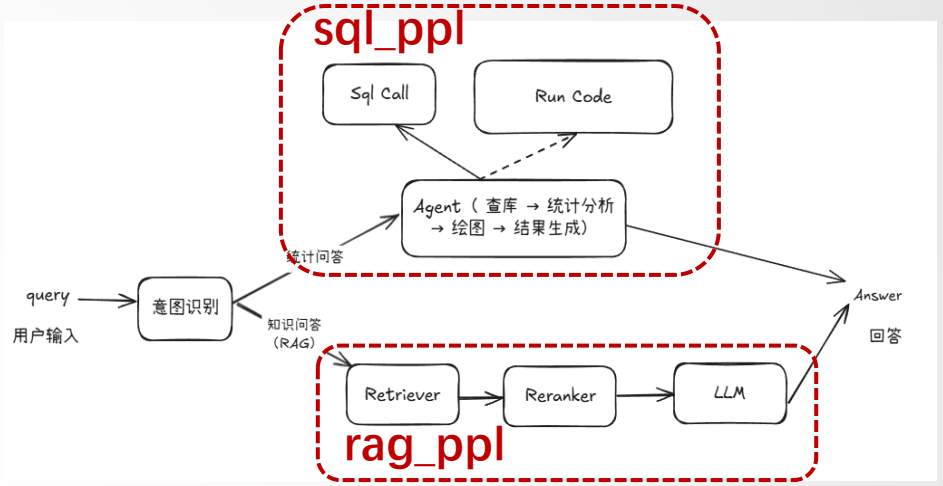
```
1.  # 导入rag pipeline 和 sql pipeline
2.  rag_ppl = build_paper_rag()
3.  sql_ppl = build_statistical_agent()

4.  # 定义意图识别模块连接rag_ppl和sql_ppl, 构建带统计分析能力的论文问答系统
5.  def build_paper_assistant():
6.      llm = OnlineChatModule(stream=False)
7.      intent_list = ["论文问答", "统计问答"]

8.      with pipeline() as ppl:
9.          ppl.classifier = IntentClassifier(llm, intent_list=intent_list)
10.         with lazyllm.switch(judge_on_full_input=False).bind(_0, ppl.input) as ppl.sw:
11.             ppl.sw.case[intent_list[0], rag_ppl]
12.             ppl.sw.case[intent_list[1], sql_ppl]

13.     return ppl
```

参照第14讲定义纯文本rag pipeline并导入

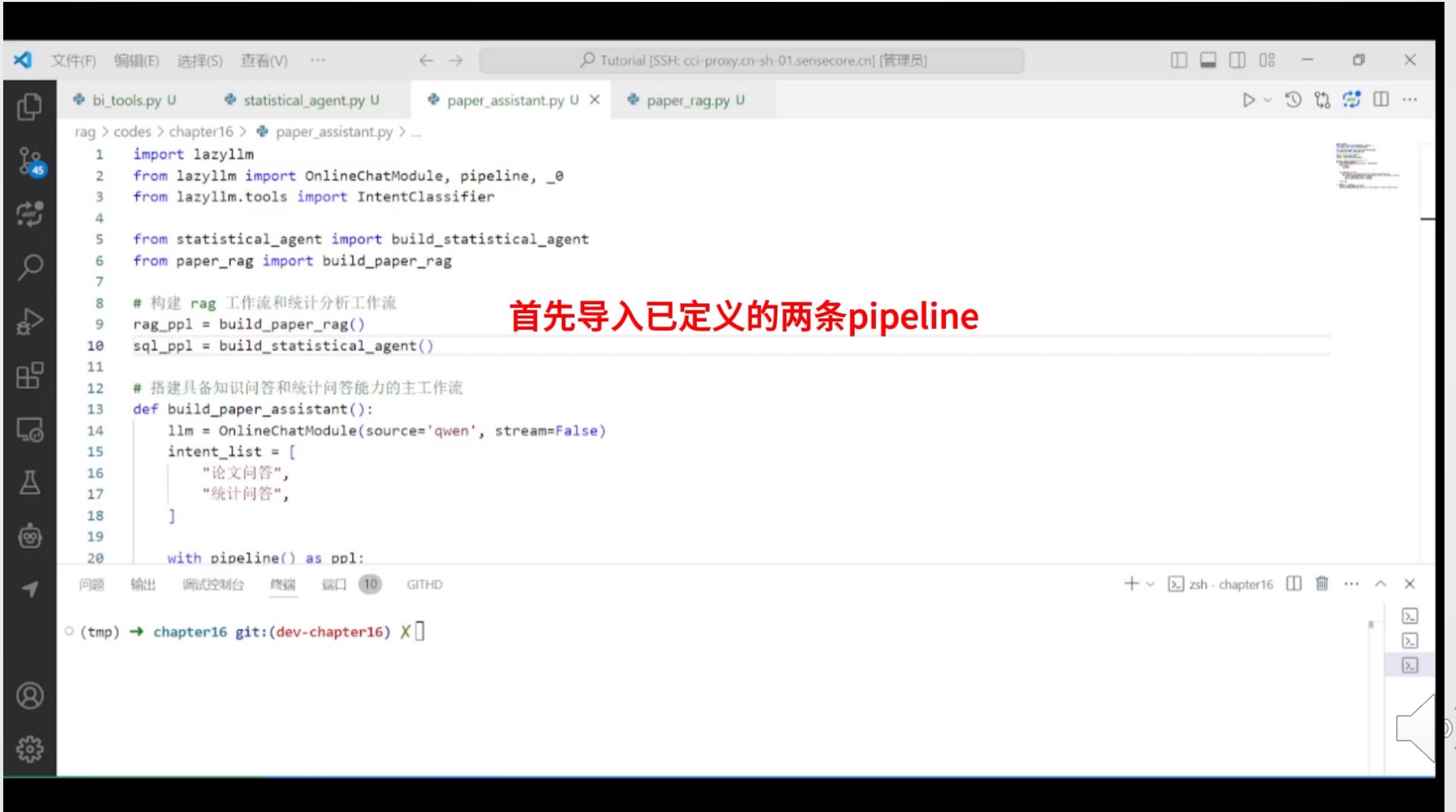
参照上一节定义sql pipeline并导入

若意图为“论文问答”，则通过rag pipeline处理

若意图为“统计问答”，则通过sql pipeline处理



运行效果


文件(F) 编辑(E) 选择(S) 查看(V) ···
Tutorial [SSH: cci-proxy.cn-sh-01.sensecore.cn] [管理员]
bi_tools.py U
statistical_agent.py U
paper_assistant.py U
paper_rag.py U
rag > codes > chapter16 > paper_assistant.py > ...
1 import lazyllm
2 from lazyllm import OnlineChatModule, pipeline, _0
3 from lazyllm.tools import IntentClassifier
4
5 from statistical_agent import build_statistical_agent
6 from paper_rag import build_paper_rag
7
8 # 构建 rag 工作流和统计分析工作流
9 rag_ppl = build_paper_rag()
10 sql_ppl = build_statistical_agent()
11
12 # 搭建具备知识问答和统计问答能力的主工作流
13 def build_paper_assistant():
14 llm = OnlineChatModule(source='qwen', stream=False)
15 intent_list = [
16 "论文问答",
17 "统计问答",
18]
19
20 with pipeline() as ppl:
首先导入已定义的两条pipeline
问题 输出 调试控制台 终端 端口 10 GITHD
zsh - chapter16
(tmp) → chapter16 git:(dev-chapter16) X

流程分析 – 简单统计问题（无绘图）

处理流程分析：

1. 用户输入query "库中一共有多少篇论文"
2. 大模型提取数据查询问题"库中一共多少篇论文" 调用SqlCall

   - input：
   "库中一共多少篇论文"
   - text2sql：
   select count(*) as total_papers from papers;
   - output：
   [{"total_papers": 100}]

3. 模型判断该结果已经可以回答query的问题，直接输出回答”库中一共有100篇论文 “。

执行效果

执行日志

```
sql_call.sql_call:163) sqlcall got query: 库中一共有多少篇文章
sql.sql_manager:222) sqlmanager got sql query: select count(*) as total_papers from papers;
```

流程分析 – 简单统计问题（有绘图）

处理流程分析：

1. 用户输入query "**根据数据库里的数据，告诉我最多的三个subject是什么并绘制柱状图**"
2. 大模型提取提取数据查询问题“找出数据库中数量最多的三个subject及其数量”调用sql call：

- input：

"找出数据库中数量最多的三个subject及其数量"

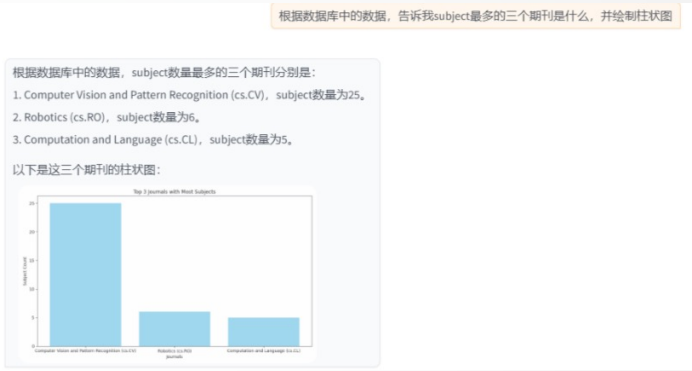
- text2sql：

```
select subject, count(*) as count
from papsers
group by count desc
limit 3;
```

- output：

[{"subject": "\nComputer Vision and Pattern Recognition (cs.CV)", "count": 25}, {"subject": "\nRobotics (cs.RO)", "count": 6}, {"subject": "\nComputation and Language (cs.CL)", "count": 5}]

执行效果



执行日志

```
4138: 2025-04-27 15:21:06 lazyllm INFO: (lazyllm.tools.sql.sql_call.sql_call:163) sqlcall got query: 找出数据库中数量最多的三个subject及其数量。
4138: 2025-04-27 15:21:08 lazyllm INFO: (lazyllm.tools.sql.sql_manager:222) sqlmanager got sql query: select subject, count(*) as count
from papers
group by subject
order by count desc
limit 3;
4138: 2025-04-27 15:21:19 lazyllm INFO: (utils.bi_tools:96) got code:
 import matplotlib.pyplot as plt
import numpy as np

# 数据
labels = ['Computer Vision and Pattern Recognition (cs.CV)', 'Robotics (cs.RO)', 'Computation and Language (cs.CL)']
counts = [25, 6, 5]

# 创建柱状图
x = np.arange(len(labels))
width = 0.35
fig, ax = plt.subplots()
rects = ax.bar(x, counts, width)

# 添加标签、标题和刻度
ax.set_ylabel('Count')
ax.set_title('Top Three Subjects')
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=45)

# 显示图表
plt.tight_layout()
image_path = './images/top_three_subjects.png'
plt.savefig(image_path)
plt.show()

image_path
```

流程分析 – 简单统计问题（有绘图）

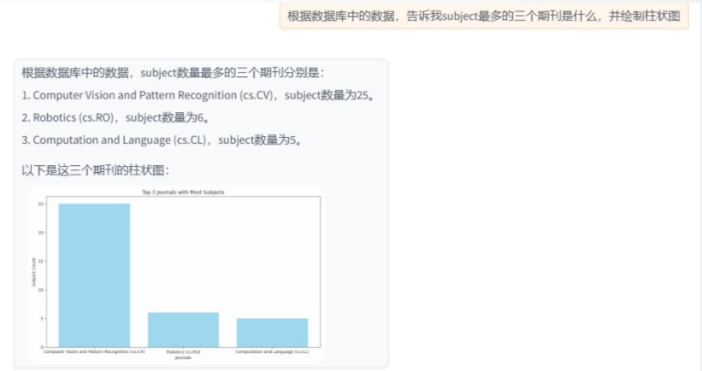
处理流程分析：

3. 模型判断该结果已经可以回答query的问题，不需要进一步进行统计分析，但是query中包含作图需求，因此其编写仅用于作图的python代码并调用run_code工具执行：

```python
import matplotlib.pyplot as plt
import numpy as np
# 数据
labels = ['Computer Vision and Pattern Recognition (cs.CV)',
'Robotics (cs.RO)', 'Computation and Language (cs.CL)']
counts = [25, 6, 5]
# 创建柱状图
x = np.arange(len(labels))
width = 0.35
fig, ax = plt.subplots()
rects = ax.bar(x, counts, width)
# 添加标签、标题和刻度
ax.set_ylabel('Count')
ax.set_title('Top Three Subjects')
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=45)
# 显示图表
plt.tight_layout()
image_path = './images/top_three_subjects.png'
plt.savefig(image_path)
```

4.作图完成后大模型根据图片路径并整合之前的结果为用户生成最终的图文并茂的回答。

执行效果



执行日志

```
4138: 2025-04-27 15:21:06 lazyllm INFO: (lazyllm.tools.sql_call.sql_call:163) sqlcall got query: 找出数据库中数量最多的三个subject及其数量。
4138: 2025-04-27 15:21:08 lazyllm INFO: (lazyllm.tools.sql.sql_manager:222) sqlmanager got sql query: select subject, count(*) as count
from papers
group by subject
order by count desc
limit 3;
4138: 2025-04-27 15:21:19 lazyllm INFO: (utils.bi_tools:96) got code:
 import matplotlib.pyplot as plt
import numpy as np

# 数据
labels = ['Computer Vision and Pattern Recognition (cs.CV)', 'Robotics (cs.RO)', 'Computation and Language (cs.CL)']
counts = [25, 6, 5]

# 创建柱状图
x = np.arange(len(labels))
width = 0.35
fig, ax = plt.subplots()
rects = ax.bar(x, counts, width)

# 添加标签、标题和刻度
ax.set_ylabel('Count')
ax.set_title('Top Three Subjects')
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=45)

# 显示图表
plt.tight_layout()
image_path = './images/top_three_subjects.png'
plt.savefig(image_path)
plt.show()

image_path
```

流程分析 – 复杂统计问题（有绘图）

处理流程分析：

1. 用户输入query "**查询数据库中所有论文的题目，基于题目内容将论文聚类为5类，每类分别列举几个文章，并用柱状图展示每类的论文数量**"
2. 大模型提取数据查询问题"库中一共多少篇论文" 调用SqlCall：

- input：

"查询库中所有论文题目"

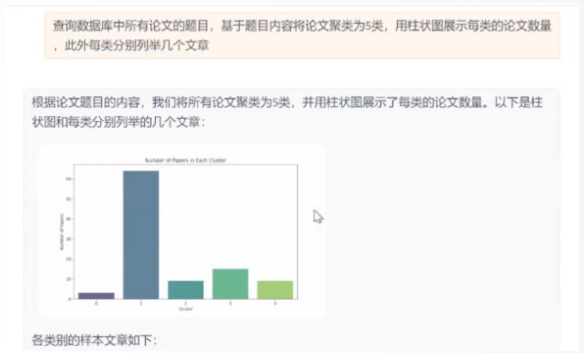
- text2sql：

select title from papers;

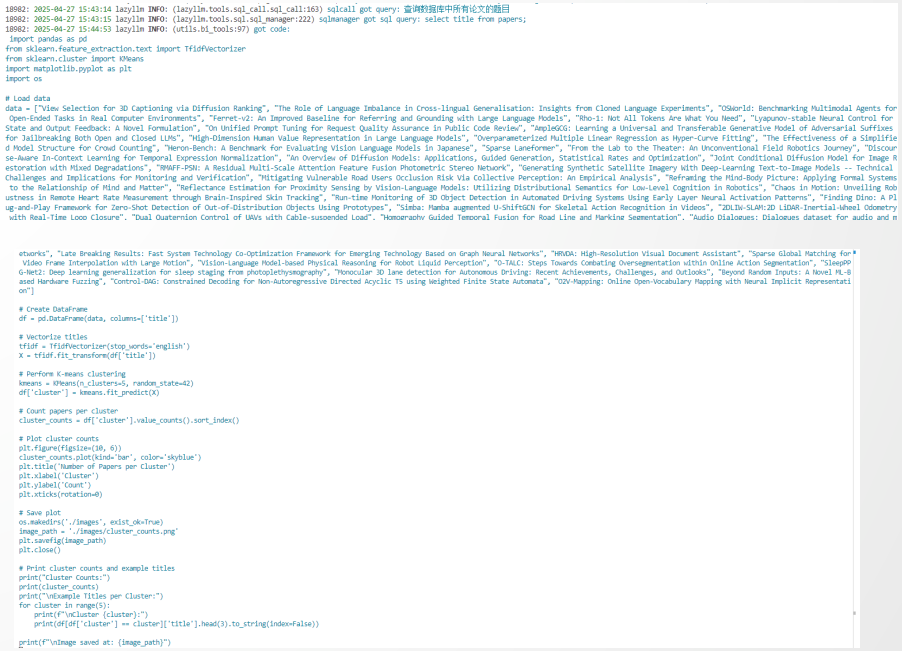
- output：

[{"title": "View Selection for 3D Captioning via Diffusion Ranking"}, {"title": "The Role of Language Imbalance in Cross-lingual Generalisation: Insights from Cloned Language Experiments"}, {"title": "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments"}, {"title": "Ferret-v2: An Improved Baseline for Referring and Grounding with Large Language Models"},…

执行效果



执行日志



流程分析 – 复杂统计问题（有绘图）

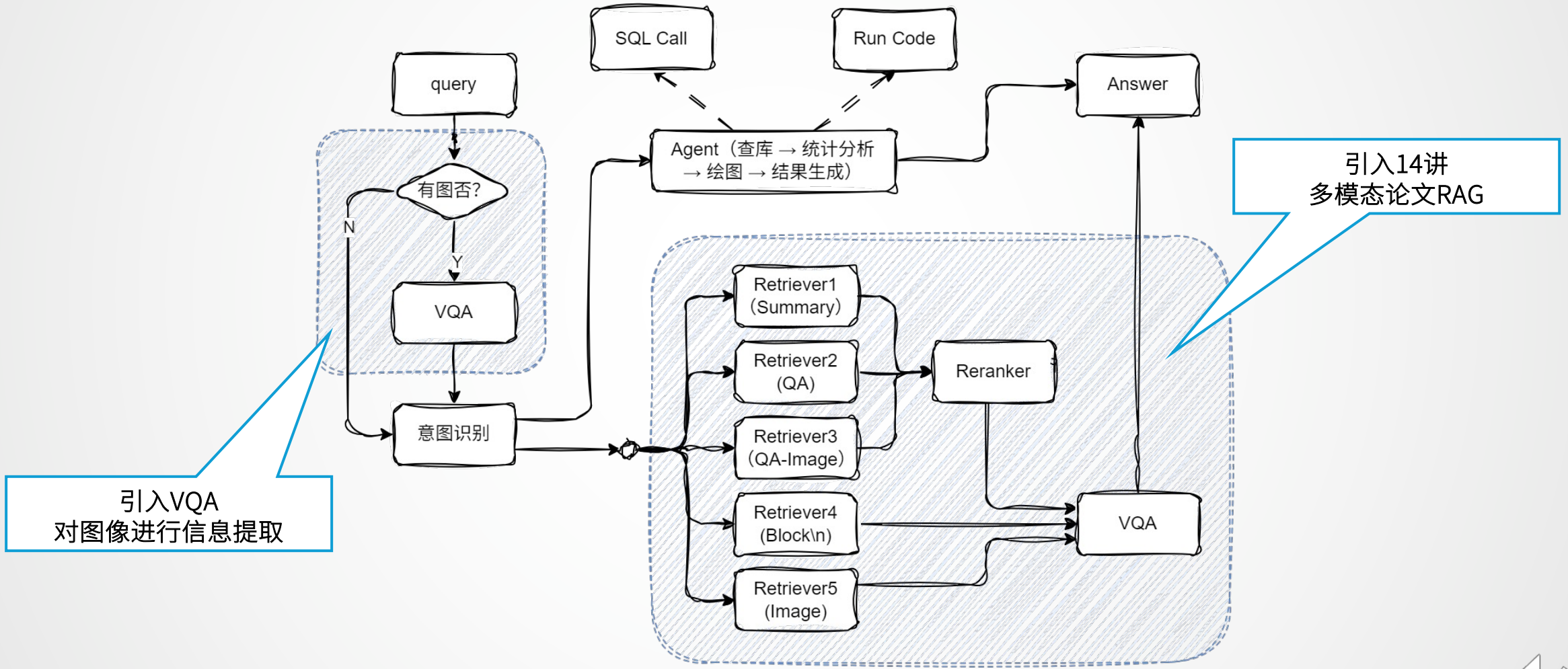
处理流程分析：

3. 模型编写程序进一步进行统计分析和绘图任务，并调用 run code 工具执行。
4. 执行完毕后大模型接收到图片路径和每类的结果，为用户生成图文并茂的回答。

```
import pandas as pd
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import os

# Load data
data = ["View Selection for 3D Captioning via
Diffusion Ranking", "The Role of..."]
# Create DataFrame
df = pd.DataFrame(data, columns=['title'])

# Vectorize titles
tfidf = TfidfVectorizer(stop_words='english')
X = tfidf.fit_transform(df['title'])

# Perform K-means clustering
kmeans = KMeans(n_clusters=5, random_state=42)
df['cluster'] = kmeans.fit_predict(X)

# Count papers per cluster
cluster_counts = df['cluster'].value_counts().sort_index()

# Plot cluster counts
plt.figure(figsize=(10, 6))
cluster_counts.plot(kind='bar', color='skyblue')
plt.title('Number of Papers per Cluster')
plt.xlabel('Cluster')
plt.ylabel('Count')
plt.xticks(rotation=0)

# Save plot
os.makedirs('./images', exist_ok=True)
image_path = './images/cluster_counts.png'
plt.savefig(image_path)
plt.close()

# Print cluster counts and example titles
print("Cluster Counts:")
print(cluster_counts)
print("\nExample Titles per Cluster:")
for cluster in range(5):
    print(f"\nCluster {cluster}:")
    print(df[df['cluster'] ==
cluster]['title'].head(3).to_string(index=False))

print(f"\nImage saved at: {image_path}")
```

RAG论文系统综合多模态方案

方案架构图



RAG论文系统综合多模态方案

主要代码实现

```python
# 构建 rag 工作流和统计分析工作流
rag_ppl = build_paper_rag()
sql_ppl = build_statistical_agent()

# 搭建具备知识问答和统计问答能力的主工作流
def build_paper_assistant():
    llm = OnlineChatModule(source='qwen', stream=False)
    vqa = lazyllm.OnlineChatModule(source="sensenova",\
        model="SenseNova-V6-Turbo").prompt(lazyllm.ChatPrompter(gen_prompt))

    with pipeline() as ppl:
        ppl.ifvqa = lazyllm.ifs(
            lambda x: x.startswith('<lazyllm-query>'),
            lambda x: vqa(x), lambda x:x)
        with IntentClassifier(llm) as ppl.ic:
            ppl.ic.case["论文问答", rag_ppl]
            ppl.ic.case["统计问答", sql_ppl]

    return ppl

if __name__ == "__main__":
    main_ppl = build_paper_assistant()
    lazyllm.WebModule(main_ppl, port=23459, static_paths="./images", encode_files=True).start().wait()
```

参照第14讲定义终极多模态rag pipeline并导入

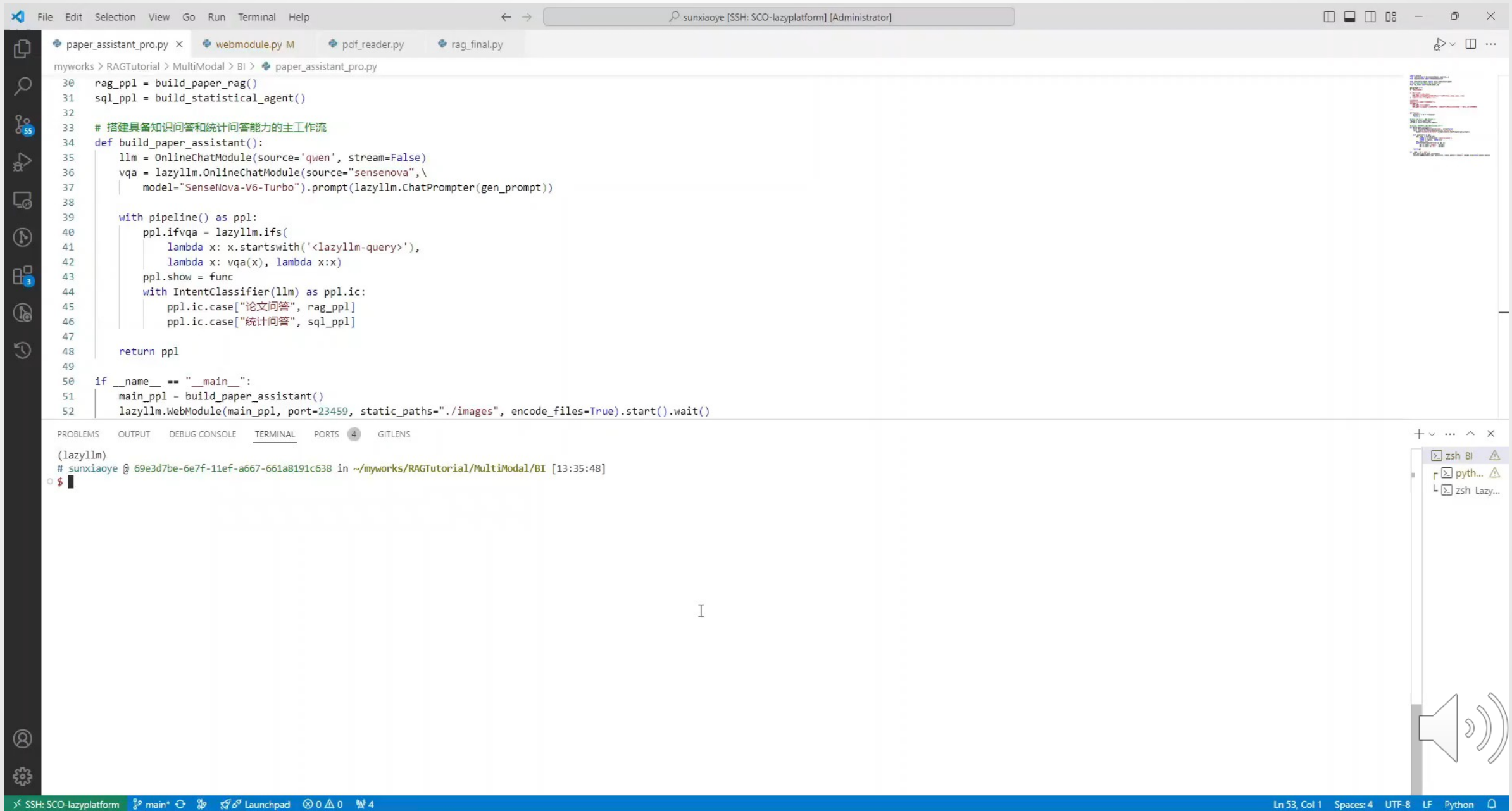
引入多模态模型提取图像信息

有图走VQA，无图就跨过VQA

若意图为“论文问答”，则通过rag pipeline处理

若意图为“统计问答”，则通过sql pipeline处理

RAG论文系统综合多模态方案

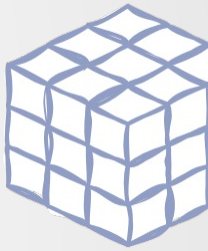


扩展阅读：从知识库构建数据库

代码实现

```python
import lazyllm
from lazyllm.tools import SqlManager

# 创建知识库
documents = lazyllm.Document(dataset_path='/path/to/kb', create_ui=False)
schema_by_llm = documents.extract_db_schema(llm= lazyllm.OnlineChatModule(source="qwen"), print_schema=True)

# 创建SQL管理器
sql_manager = SqlManager(db_type="SQLite", user=None,
   password=None, host=None, port=None, db_name="doc.db")
# 配置提取字段
refined_schema = [
    {"key": "document_title", "desc": "The title of the paper.", "type": "text"},
    {"key": "author_name", "desc": "The names of the author of the paper.", "type": "text"},
    {"key": "keywords", "desc": "Key terms or themes discussed in the paer.", "type": "text"},
    {"key": "content_summary", "desc": "A brief summary of the paper's main content", "type": "text"},
]
# 知识库连接到SQL并设置待提取字段
documents.connect_sql_manager(
    sql_manager=sql_manager,
    schma=refined_schema or schema_by_llm,
    force_refresh=True,
)
# 开始提取知识库到数据集
documents.update_database(llm=lazyllm.OnlineChatModule(source="qwen"))
# 展示提取到的内容:
str_result = sql_manager.execute_query(f"select * from {documents._doc_to_db_processor.doc_table_name}")
print(f"str_result: {str_result}")
```

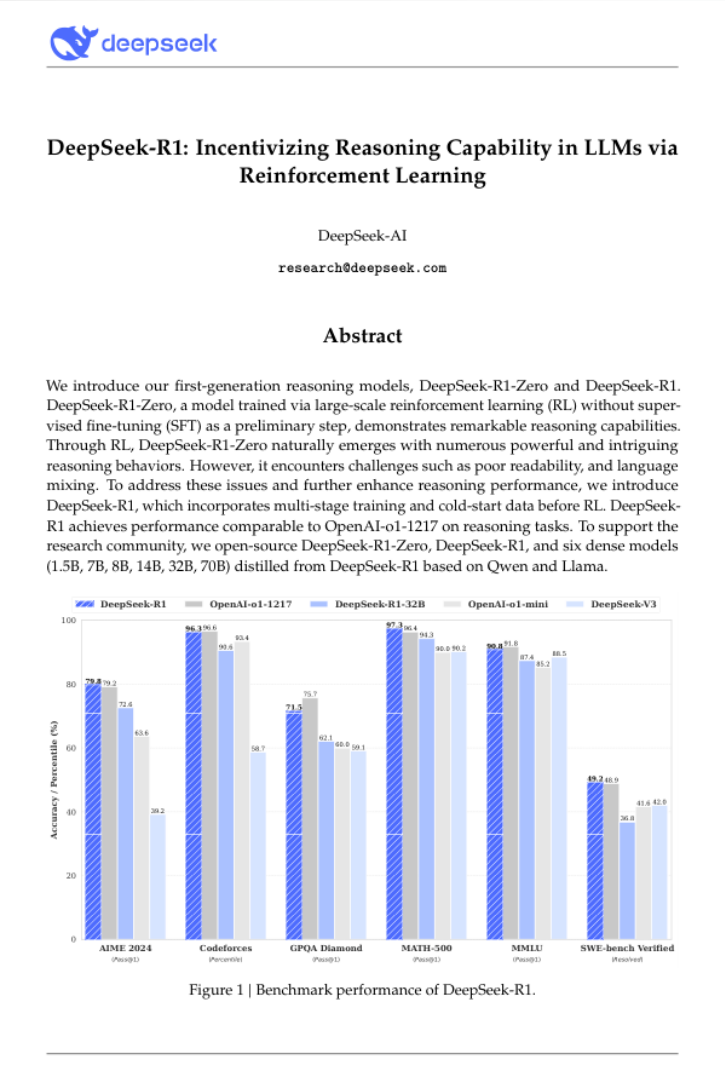
知识库



数据库

扩展阅读：从知识库构建数据库

效果展示



deepseek

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Abstract

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.

Figure 1 | Benchmark performance of DeepSeek-R1.


```
[
    {
        "lazyllm_uuid": "2c42c181-bc26-43ca-a822-505b22f5d19e",
        "lazyllm_created_at": "2025-05-16 19:10:12.014368",
        "lazyllm_doc_path": "/path/to/pdfs/DeepSeek_R1.pdf",
        "document_title": "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning",
        "author_name": "DeepSeek-AI",
        "keywords": "DeepSeek-R1-Zero, DeepSeek-R1, reinforcement learning, reasoning models, multi-stage training, open-source, dense models, distilled models, Qwen, Llama",
        "content_summary": "The paper introduces the first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero is trained using large-scale reinforcement learning without supervised fine-tuning and demonstrates strong reasoning capabilities but faces challenges such as poor readability and language mixing. To improve upon this, DeepSeek-R1 incorporates multi-stage training and cold-start data before reinforcement learning, achieving performance comparable to OpenAI-o1-1217 on reasoning tasks. The research community is supported by the open-sourcing of DeepSeek-R1-Zero, DeepSeek-R1, and six dense models distilled from DeepSeek-R1 based on Qwen and Llama."
    }
]
```

感谢聆听

Thanks for Listening